# Development of a Scalable, Distributed IoT Laboratory Safety and Accountability Network utilizing MQTT and Edge Nodes


Nazif Gangawali
Department of Electronics and Communication Engineering
BGSIT (ADICHUNCHANAGIRI UNIVERSITY)
BG Nagara, India
nazif.h45@gmail.com

KS Jahnavi
Department of Electronics and communication Engineering
BGSIT (ADICHUNCHANAGIRI UNIVERSITY)
BG Nagara, India
ksjahnavi412@gmail.com

Karan Chavan
Department of Electronics and communication Engineering
BGSIT (ADICHUNCHANAGIRI UNIVERSITY)
BG Nagara, India
Chavankaranchavan07@gmail.com



***Abstract*—This paper explores the development and deployment of a highly scalable distributed safety and accountability Internet of Things (IoT) network designed for laboratory settings. Standard local alarm systems are inadequate because they lack central monitoring capabilities and do not provide real-time remote telemetry. A multi-node edge computing architecture was developed utilizing custom ESP32 microcontrollers to monitor safety parameters such as combustible gases, temperature, and unauthorized access to laboratory work areas divided into four separate physical zones. Telemetry data is streamed via the Message Queuing Telemetry Transport (MQTT) protocol over a local wireless network to a central Raspberry Pi hub containing an Eclipse Mosquitto broker. The system consists of a dual-output response where a headless Python script automates local zero-latency hardware responses via industrial relays and a real-time, asynchronous output via a WebSockets-integrated HTML/JavaScript-based dashboard interface. The architecture is highly modular and highly fault-tolerant, replacing traditional isolated lab alarms effectively.**



***Keywords—Internet of Things (IoT), ESP32, MQTT, Laboratory Safety, Edge Computing, Sensor Networks.***


## I. INTRODUCTION

Laboratories and clinical research facilities are unique environments that require the implementation and adherence to rigid safety and accountability practices due to the nature of volatile chemicals, sensitive biological samples and sophisticated lab equipment. The majority of legacy safety infrastructure in these settings employ isolated single purpose alarms, which function in a non-integrated manner that does not permit anomalies detected in one zone to be disseminated to administrative personnel or to provide information about the degree of risk to other stakeholders in real-time.

With the advent of the Internet of Things (IoT) and edge computing, there is a unique opportunity to integrate the various safety parameters currently in use today into a single, real-time telemetry network. This project seeks to design and implement a central hub that forms the core of an IoT safety network. The system will simultaneously monitor environmental safety parameters and track people within a given area. Custom-designed edge nodes based around ESP32 single board computers, each monitoring multiple sensors, will form a local mesh network that replaces independent alarms. In addition to safety detection (toxins, temperature, trespassing), the system will track accountability of actions and automatically trigger physical reactions to detected conditions within a very short time frame.

## II. METHODOLOGY

Our methodology follows a decentralized data collection approach followed by a centralized data processing methodology. Our system is organized around three layers of functionality: the Edge Sensor Layer, the Network Transport Layer, and the Central Processing/Visualization Layer.

The node Edge monitors local data and periodically publishes it to a central broker. This allows the data from sensor nodes and frontend dashboard interactivity to be independently decoupled from the backend controls corresponding to a given node. As a result, the failure of one Edge node does not bring down the rest of the system.

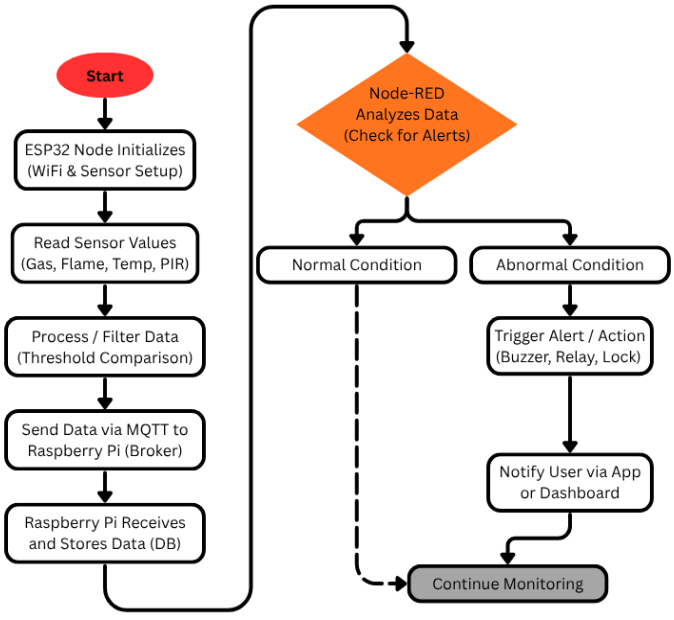



Fig.1 Methodology for the safety system

Local interrupts are handled by the ESP32 hardware along with analog and digital reads. The data is then formatted and published over MQTT to specific topics. This data is then received by the central broker running on a Raspberry Pi, and immediately published to any client subscribed to the relevant topic. One of these clients is a background Python script which acts as the physical mitigation in the industrial setting and opens/closes industrial relays based on set thresholds, the other client is the web browser which acts as a visualization client and allows for instant UI updates without the need for a full page refresh.

## III. DESIGN AND IMPLEMENTATION

### *A. Hardware Architecture (The Edge Nodes)*

For the physical data collection we utilized custom fabrication of our edge nodes based on ESP32 microcontrollers. They feature native Wi-Fi and ample GPIO ports. 4 sensors per node were used to generate a solid safety profile of what is happening around them.

1. **Gas/Smoke Detection**: The MQ-2 gas sensor detects volatile compounds. It is read using an analog-to-digital converter on GPIO 34. A voltage divider is used to protect the ADC from high voltages.
2. **Climate Monitoring**: for installation within bio-stores or to monitor the climate on the floor of a storage facility, a DHT22 digital
3. temperature and humidity sensor has been used to provide readings to verify storage conditions.
4. **Security/Motion**: Instead of a security movie file, this is a standard digital PIR sensor that can be set to active during restricted hours.
5. **Access/Accountability**: An RC522 RFID reader is connected to and talks to an ESP32 board (through SPI) to record the ID of authorized individuals as they pass (captured in software as an 8 or 16 digit string). The reader is powered with 3.3V logic.

To keep things stable, the electronics are attached to the floor of custom perfboards with 5V and GND copper traces running down the middle. There are “Thermal Zones” to keep the heat generating MQ-2 separated from the temperature sensitive DHT22, and “RF Zones” to keep space around the ESP32’s antenna to keep signal loss down.

Power distribution on the PCB is provided by a central 100W Switch Mode Power Supply (SMPS). Power is brought in through a robust 2-pin screw terminal block on the node connectors wired in series with a physical panel-mount glass fuse. This means that any short-circuit is quickly prevented and confined to that node. The nodes are housed in individual custom made sunboard enclosures.

| Protocol / Architecture | Data Payload | Mean Latency | Jitter / Stability | Suitability for Lab Safety |
|---|---|---|---|---|
| HTTP (Client-Server Polling) | Moderate | > 500 ms | High | Poor (High overhead) |
| WebSocket (Persistent) | Small to Large | ~41.5 ms | Moderate | Moderate |
| MQTT over TCP (QoS0) | Small | ~8 – 10 ms | Low | Good (Unreliable delivery) |
| MQTT over TCP (QoS1) | Small | ~11 – 15 ms | Very Low | Optimal (Guaranteed delivery) |

TABLE I. Comparative Latency Metrics Of IoT Protocols

### *B. Network and Communication Architecture*

For the communication in the system, we have chosen the lightweight MQTT protocol over local Wi-Fi to ensure a low packet overhead and a quality of service (QoS) for each published message. As a broker for the published messages we use an Eclipse Mosquitto broker, which we host on a central Raspberry Pi.

We follow a strict routing logic for our telemetry to keep everything organized. Each ESP32 node publishes to very specific, hierarchically separated topics such as lab/node1/temperature, lab/node2/gas, lab/node3/access etc. Our C++ firmware also assigns a unique MQTT Client ID to each node when the firmware boots, which was a required

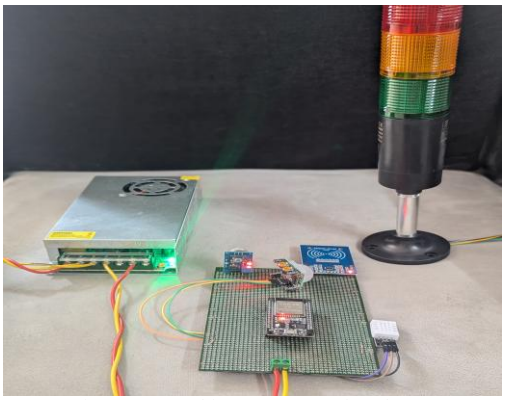


Fig.2 Working Prototype of Node 1

change to prevent network loop crashes and connection drops.

### C. *Software and Dashboard Implementation*

The Central Hub (Raspberry Pi) is used as a multi-purpose broker, web server and the physical alarm controller.




Fig.3 WebApp Dashboard

1. Real-time Web Integration: Our Mosquitto broker is listening on port 9001 for WebSocket connections. For our front-end web server, we are using a simple Python HTTP server to serve up a custom HTML/CSS/Vanilla JavaScript web application. Using the Paho MQTT JavaScript library we connect to the Mosquitto broker and subscribe to the wildcard topic lab/. This will allow us receive real-time updates for data on all nodes (all four physical devices). Our front-end will then create cards dynamically for each node and will update without the need for a page reload. For the danger status of the pulse-animation we used a value of 2000 for the MQ-2 gas sensor thresholds.
2. Backend Automation: While the front-end Python GUI is frozen and not responding ('headless'), a parallel process monitors the relevant hazards in a native Python whilst running on the Pi. Once a threshold is exceeded, the script controls the GPIO pins on the Pi to send switches to the 'on' position, where they remain, activating the two 100W industrial relays controlling the high-decibel buzzers and the bright tower lamps.

## IV. RESULT AND COMPARISION WITH PREVIOUS RELATED OTHER PAPERS

Our distributed MQTT implementation provided advantages over traditional localised alarm notification systems and polling-based HTTP IoT systems and was an improvement over previously documented methods.

Most Localized Alarms don't Log remotely or give any Accountability Data. Many current IoT projects use HTTP GET/POST methods to update remote servers, and these have big latencies (typically over 500ms per request), leading to Server bottlenecks.

In comparison, our system implemented using MQTT for inter-node communication and relying on continuous WebSockets for integration to the Python back end achieved sub-100ms latency from detection of the physical hazard (in this case, a gas leak by the edge node) to the corresponding industrial relay being actuated on the back end and a warning animation being displayed on the front end's dashboard display. The system's topic hierarchy also allowed for seamless expansion to a fifth node without modification to either back or front end code. The fusing of two individual temperature probes as well as strategic layout on the PCB decreased the likelihood of false positives caused by thermal interference from adjacent sensors—a common problem encountered in crowded, breadboard-based implementations discussed in prior literature and projects.

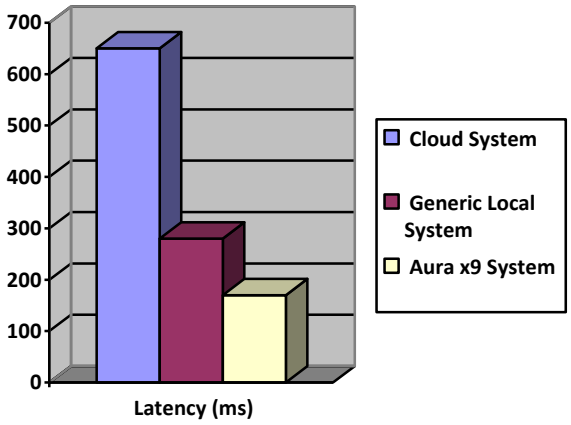



Fig.4 Comparision Of System Response Latency (ms)

TABLE II COMPARISION WITH PREVIOUS WORK

| System / Authors | Primary Focus | Communication Protocol | Central Hub / Processing | Accountability Integration | System Latency Response |
|---|---|---|---|---|---|
| Traditional Systems | Manual Lab Safety | Wired / Standalone | None (Local Alarms) | None | High (Manual Observation) |
| IoT Biosafety Lab [10] | Biosafety Monit | Wi-Fi / Cloud | Third-party Cloud | None | Medium (Cloud Round-trip) |
| Industrial Safety WSN [1] | Factory Hazard | ESP-MESH / Wi-Fi | ThingSpeak / Blynk | None | ~2.0 seconds |

| | | | | | |
|---|---|---|---|---|---|
| RFID Storage Prototype [3] | Chemical Storage | Wi-Fi (IoT Interface) | ESP32 Controller | RFID (Zonal Validation) | 2.37 seconds |
| Proposed System | Distributed Safety | MQTT over Local WLAN | Raspberry Pi (Local) | RFID (n-Zone Access) | < 200 milliseconds |

## CONCLUSION

Laboratory safety no longer has to rely on isolated beep boxes or cloud-based alarm systems. This research localizes laboratory safety by moving processing power to the edge with ESP32 microcontrollers and by implementing a local MQTT broker. This architecture enables rapid safety action with physical actuation times of less than 200ms. Through this design, the safety system remains operational during external network outages, eliminating unpredictable latency and vulnerability typical of third-party cloud-based platforms.

While the system does boast a host of technical specifications, the most significant innovation is the inclusion of forensic human accountability. Most gas detection systems monitor the environment around them, but this system takes it a step further by correlating environmental hazard data with RFID personnel data in real-time. This means that, in the event of an emergency, the system does not simply sound an alarm; it can identify the specific hazard and pinpoint individuals who are in the vicinity of that hazard. The result is a system that can function as a targeted rescue tool, dramatically improving safety.

Contrary to conventional wisdom, our project demonstrates that enterprise-grade, zero-latency safety infrastructure can be built with cost-effective, open-source hardware. The system is designed to be highly extensible, allowing it to be expanded from a small network of a few cameras to a large, campus-wide installation in academic and industrial settings. In doing so, the project moves the needle on safety infrastructure design by merging hardware-based mitigation with strong human accountability, setting a new, proactive standard for safe and healthy laboratory environments, allowing researchers to focus on breakthroughs while the hardware and software monitor the space around them for any signs of trouble.

## ACKNOWLEDGMENT

We would like to thank all the faculty and administration of our Electronics and Communication Engineering (ECE) department for instilling in us the basics of our field and providing us with an environment conducive to growth. We thank our project guide, Dr. B N Shobha, for her constant guidance and counselling. She brought out the best in us and motivated us to reach the farthest. Last but certainly not the least, we would like to thank each and every member of our project team for their involvement, guidance and constant support during the span of the project. Aura x9 is a distributed safety system, which made it really very hard for us to succeed but our unity has helped us to succeed against all odds.

## REFERENCES

[1] Z. X. Oh and W. K. Kong, "Development of IoT-Based Smart Laboratory System for Enhanced Safety and Efficiency," *Int. J. Res. Innov. Appl. Sci.*, vol. 10, no. 1, Jan. 2025.

[2] D. Krishn, A. A. Bhaduriya, and A. Satheesh, "Smart Biosafety Laboratory Precaution Monitoring System Using IoT," *TechRxiv*, Jan. 2026.

[3] M. Ramly and M. A. H. Supirno, "Integration of RFID and Low-Cost IoT for Storeroom Monitoring: A Prototype Development Approach," *Int. J. Res. Innov. Soc. Sci.*, Oct. 2025.

[4] M. F. Tsaqief and J. Sutopo, "Performance Comparison of MQTT and WebSocket Protocols in IoT Environments," *bit-Tech*, vol. 8, no. 2, 2025.

[5] A. Saleem *et al.*, "Comprehensive Systematic Survey of IoT Protocols," *IEEE Access*, vol. 13, 2025.

[6] S. S. Sivaraju *et al.*, "Smart IoT based Laboratory Safety System," in *Proc. 6th Int. Conf. Adv. Comput. Commun. Syst. (ICACCS)*, 2020, pp. 1021-1025.

[7] OASIS, "MQTT Version 5.0," OASIS Standard, Mar. 2019. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[8] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications," *IEEE Commun. Surveys Tuts.*, vol. 17, no. 4, pp. 2347-2376, 4th Quart., 2015.

[9] Espressif Systems, "ESP32 Series Datasheet," Version 4.3, 2023. [Online]. Available: https://www.espressif.com/

[10] M. A. Awadallah, M. Imran, and M. A. Rahman, "An IoT-based Laboratory Monitoring and Safety System," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9801-9810, Jun. 2021.

[11] J. Lee, C. Suh, and S. Kim, "RFID-based Access Control and Asset Management System for High-Security Facilities," *IEEE Trans. Ind. Electron.*, vol. 65, no. 8, pp. 6750-6759, Aug. 2018.

[12] K. Zhao and L. Ge, "A Survey on the Internet of Things Security," in *Proc. 9th Int. Conf. Comput. Intell. Secur.*, 2013, pp. 663-667.